Thrive With AI

Problem Setting

This project has three entirely different AI algorithms that take control of life as it evolves from a single organelle single-celled organism up to but not including multicellular life via the open-source game Thrive! Thrive is an evolution game akin to Spore, where the goal of the single-celled stage is to have your species survive 20 generations where you compete against different species that stem and evolve based on your original species in an ever-growing tree. The environment changes dynamically based on how life interacts with it, producing and using different resources changing which strategies work overtime. To survive a generation the population of the player species must be greater than 50, and to win at generation 20 it must be greater than 300. The single celled stage is split up between two sections: “active play” where the player plays as the micro-organism in the world, and the editor view where the player’s micro-organism can be modified between each “active play.” To skip “active play” cheat mode is activated at the beginning of the run with Infinite Compounds and Unlimited Growth Speed activated so the cell does not need to collect any compounds and can quickly re-enter the editor (simulations figure out changes in the environment and populations even if the player does nothing in “active play”.)

Figure 1: Starting point for all AI algorithms. In “active play” between editor instances.

This project will constrict the problem setting to the editor view. In the editor view, one can see the evolutionary tree that shows a tree of all the different evolution of species that evolve off each other (cyan nodes) starting from your species and can die off (red nodes). It also shows the atmospheric gasses, compounds, and populations of all species as they change over time.

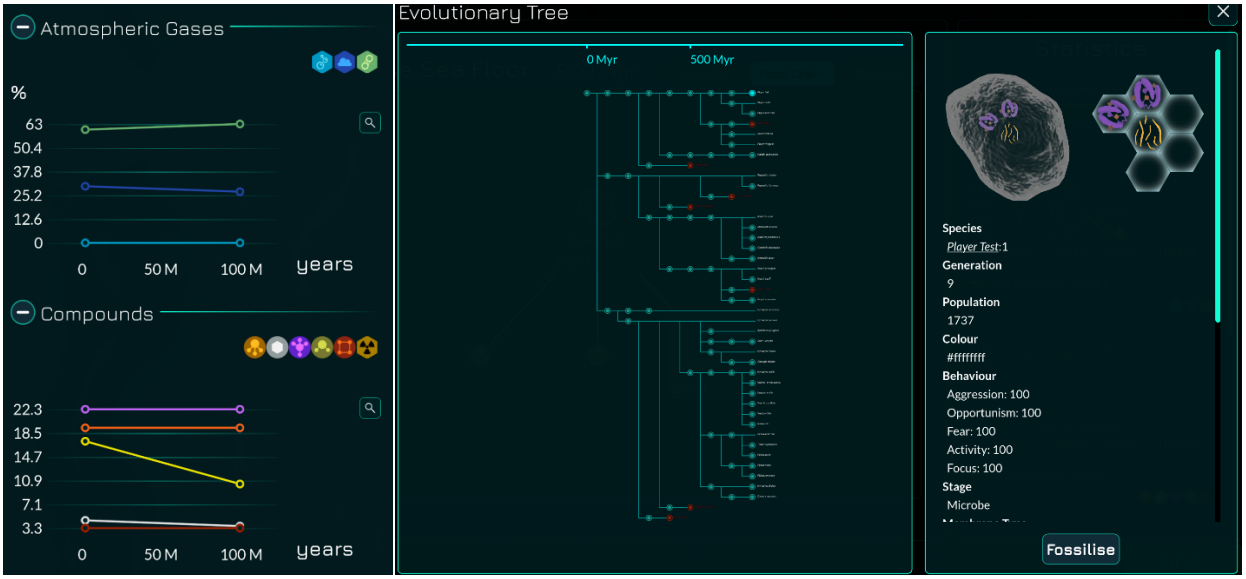

Figure 2: Evolutionary Tree and Statistics tabs in the editor view.

As this game is open source, I have modified it to output the following values to a log text file every time the editor is opened:

Ammonia, Glucose, Phosphates, Hydrogen Sulfide, Oxygen, Carbon Dioxide, Nitrogen, Sunlight, Temperature, Iron, Speed (player species/micro-organism speed), ATP Production, ATP Consumption, and Population.

These values are then converted into a unique .csv file for each of the AI algorithms to use as inputs.

To reduce complexity, the AI algorithms will be forced to stay in the starting biome (volcanic vents). There are many smaller options to tweak in the player species's Behaviour, Membrane, Tolerance, and Growth Order. But these have a small effect on the player species and are not very important if the player stays in the same biome as this problem setting will be. So, to further reduce complexity the AI algorithms will only be able to add organelles to the player species, with 12 options to pick from (excludes the nucleus and organelles that require it for now.)

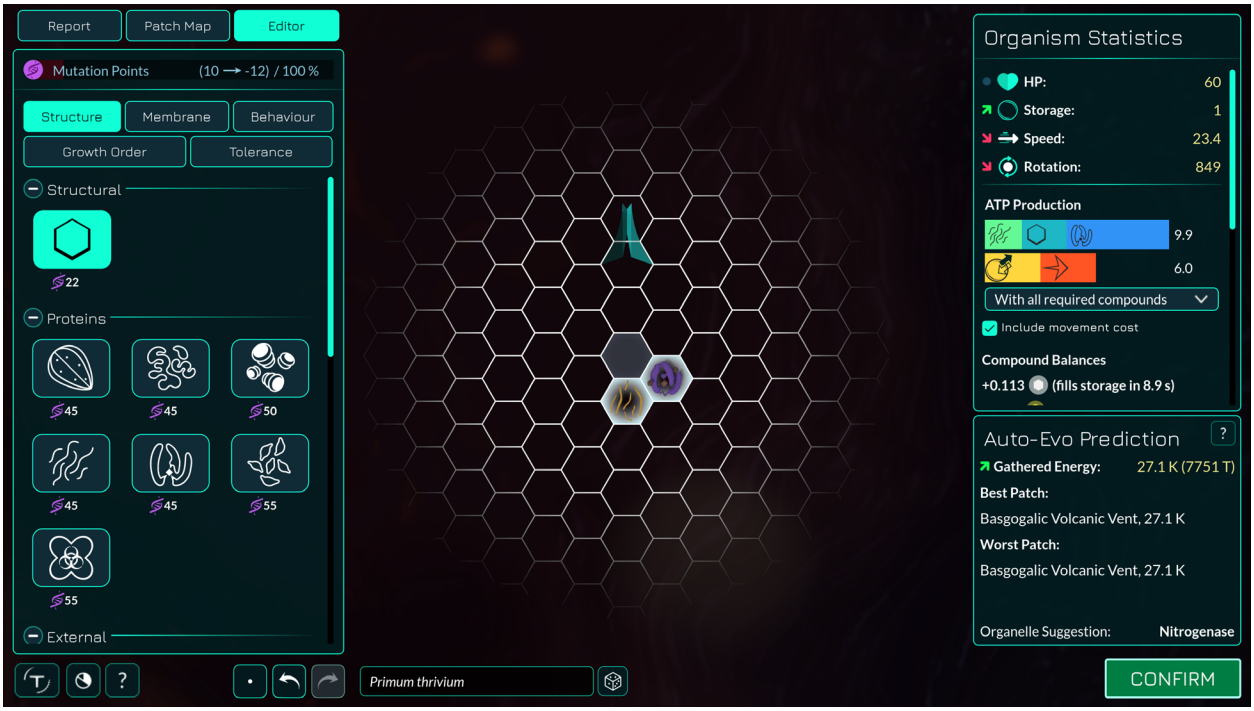

Figure 3: Main editor view where the AI algorithms will be operating.

To interact with the game, I made an interface using pyautogui and pydirectinput to take control of the user's mouse and keyboard with exact locations of each in-game button and organelle location based on a 16:9 4K monitor. The AI algorithms pick either none, one, or more organelles depending on the algorithm, placing them in a circle around the starting cytoplasm as pictured in the center of Figure 3.

Definitions:

MP = Mutation Points. There are 100 Mutation Points usable in a given generation.

CO_2 = Carbon Dioxide.

Active stage = The stage between editor instances. Where the cell can be played and the player must gather/make enough Phosphates and Ammonia for mitosis so they can reach the editor. We skip this stage by enabling cheats and set infinite compounds and unlimited growth to on. Other than allowing the player to survive no matter the microbe they have built, this will not affect the results.

Methods

Numeric Planner (ENHSP)

The planner agent uses ENHSP-2020 as the planner and parser. This is a numeric planner that was required as the majority of the information the planner needs to use within the problem setting is numeric in nature, with values ranging from extremely small decimals up to numbers above 10000, which ENHSP-2020 is great at handling. The planner takes in the current compound values in the problem file, then finds a plan via a series of actions. There are 12 actions in total, and each action is an organelle to select. The functions include every compound, a preferencer function (not preference as it gave a syntax error, likely a variable name assigned to something internal), an organelles function, and a total function. Preferencer is how preferable the current plan is, organelles is the current count of the number of organelles on the microbe, and total is the total cost of the plan. Total cost must be equal to or less than 100 as the maximum number of MP that can be spent in a generation is 100. There is also a list of predicates stating which organelle(s) have been selected. Due to this only one of each type of organelle can be selected to be added in a generation, but this will lead to more unique microbes so is worth studying. Each action uses these functions and predicates to determine if the action can be selected (if the current total cost + the cost of the organelle will be over 100, and any other restrictions). If so, the total increases by the cost of the organelle, the number of organelles is increased by one, and the functions are used to determine how much the preferencer will increase and decrease. The plan is working toward a goal of having the total cost be less than or equal to 100, with a metric to maximize the peferencer, which is ultimately the main goal of the planner (though not technically a “goal”).

There is a planner_player.py file that creates a new problem file for the planner each generation. The planner makes a plan for one generation at a time, so only needs the info from one generation at a time. There is a lot of math involved in effectively predicting the values of future generations, which is why the planner only plans one generation at a time.

Bayes Net

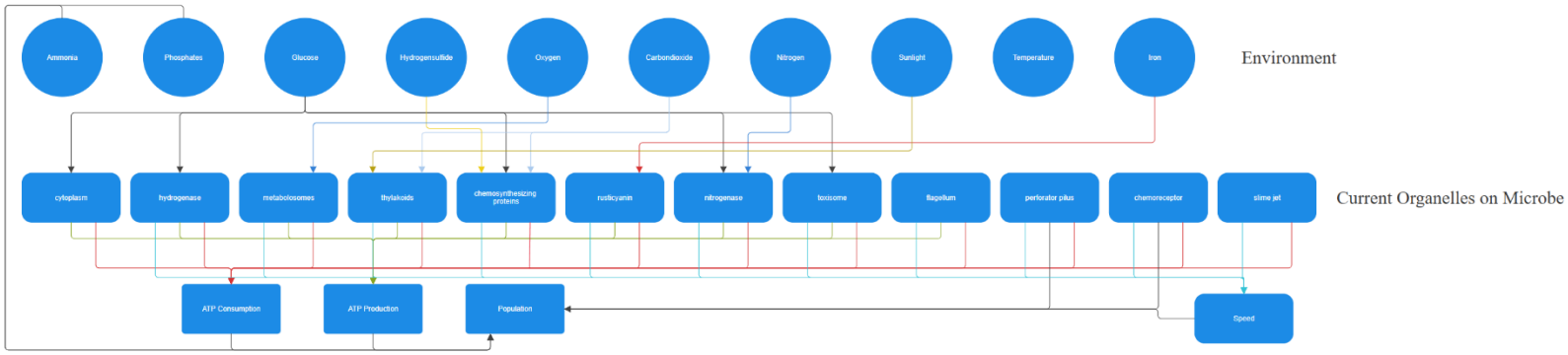

Figure 1: Original Bayes Net design.

Originally, I had designed the Bayes Net as close as possible to what I thought each variable effected. It involved an environment layer that took in all the compound data from each run, leading into an organelle layer which would simultaneously be what organelles were currently on the microbe and what the Bayes net would use to predict what organelle to add next. The connections between the first layer and the second layer were based on what compounds each organelle used/required. For example, in Figure 2, Chemosynthesizing Proteins make ATP out of hydrogen sulfide at a rate scaled with CO_2, as well as glucose.

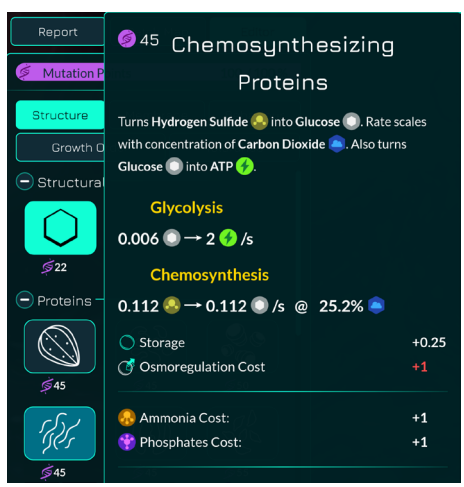

Figure 2: Chemosynthesizing Proteins info panel.

The third layer involved any higher-level processes such as speed, which is lowered as the microbe adds organelles causing it to grow in size and can be increased by specific organelles like flagellum and slime jets. Similarly, every organelle added requires ATP to operate and stay alive, some requiring more than others depending on the complexity of its actions, but most have a baseline movement cost and osmoregulation cost like the one

seen below “Storage” in Figure 2. This is why all organelles are connected to ATP Consumption. ATP Production was connected only to organelles that directly produced ATP, or that made compounds that it can then use to make ATP. This can be seen in Figure 2 where Chemosynthesis uses hydrogen sulfide to make glucose at a 1:1 ratio, then uses that glucose to make ATP. Organelles that indirectly made ATP via assisting in killing other microbes then consuming the remains such as Perforator Pilus were not considered as they did not change the ATP predicted production value the game shows in the editor, which is what was used in the data the Bayes Net references.

Within the third layer ATP Consumption and Production are directly linked to Population as they control how much energy a microbe can produce and consumes during its lifetime, and so if the balance is off and the microbe only barely produces enough ATP at its peak to offset the ATP consumption, or if the ATP consumption is higher than peak production, the microbe’s lifetime and thus population will be cut short. Speed is connected to population as a faster microbe can take advantage of resources across a larger area, and can avoid predators and/or catch prey much easier.

Finally, Ammonia and Phosphates are the only environment compounds connected to Population as they are the two compounds needed for mitosis so the population can grow. ATP is used to make these compounds internally, but to speed up the process having more Ammonia and Phosphates in the area will allow the microbe populations in the area to grow faster. Temperature is not connected to anything as it does not have a direct impact on organelles or population and does not typically change in the starting biome. It was left in the diagram and the Bayes Net data as extra data in case I wanted to expand what the Bayes Net and the other agents were able to modify, as temperature tolerance is an option that costs MP to change in a generation.

Ultimately, this model of the Bayes Net is not very practical and did not work out as there are nodes that have up to 12 connections into them, like ATP Consumption having the entire second layer connect to it, which would then mean at a minimum it would require 2^{12} good, unique data points at a minimum to run properly. As the game needs to be run graphically to gather each data point, gathering 4096 data points would take so long it would likely need to be run for 12 hours a day, 102 days straight on one device, with me switching the seed every 10-20 generations so it keeps getting good data. Another issue with this model comes with the implementation. It predicted which organelle to add by getting the data of the current generation, then looping through 12 times, removing the data from each organelle one at a time such that only one is missing at a time, then predicting what each value should be. Based on that, it would then pick the largest value out of all of them and say that the Bayes Net chose that to be what to add. The issue with this is that it

would end up predicting exactly what organelles is on the microbe currently, not what it should be next, and so the agent would end up picking the organelle that has the highest number of organelles currently on the microbe. This would end up making a giant blob of cytoplasm as that is the organelle that starts on the microbe initially.

So, a solution to all these problems! Step 1, remove any and all unnecessary nodes from the Bayes net:

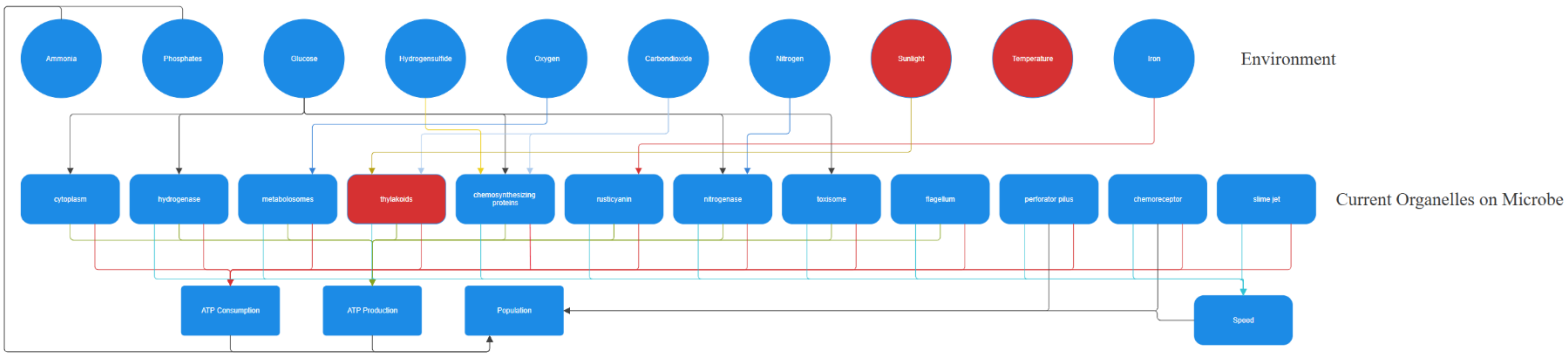

Figure 3: Original Bayes Net design with unnecessary nodes highlighted in red.

Thylakoids are not a necessary option for any of the agents as it is like a simplified version of the chloroplast so requires sunlight to function. As the starting biome is deep underwater in a volcanic vent far from any sunlight, this will always be a suboptimal option. As such we do not need sunlight in the net, nor temperature as discussed earlier.

To fix both issues in one, we can compress the entire second layer into one node that keeps track of what organelle to add next. A number from 0 to 12 where 0 is cytoplasm, 11 is slime jet, and 12 is nothing (i.e. chooses best option is to not add anything). This does remove all information of what the microbe currently looks like but shrinks the maximum connections to any given node to 9, so $2^9 = 512$ data points to gather. This is much more achievable but still not quite enough to be able to do within a reasonable length of time (likely 10 days if it is run optimally). The new Predictor node is directly connected with all compounds that was connected to the previous second layer, but also has population, ATP Consumption and ATP production feeding back into it, as the decision is made based on all of that data. ATP Consumed and Produced is connected to it to make up for some of the data lost by removing all the organelle counts that are currently on the microbe. Speed is not entirely necessary which is why it is highlighted in red in Figure 4, but as it does not increase the largest minimum number of data points to gather, it can stay.

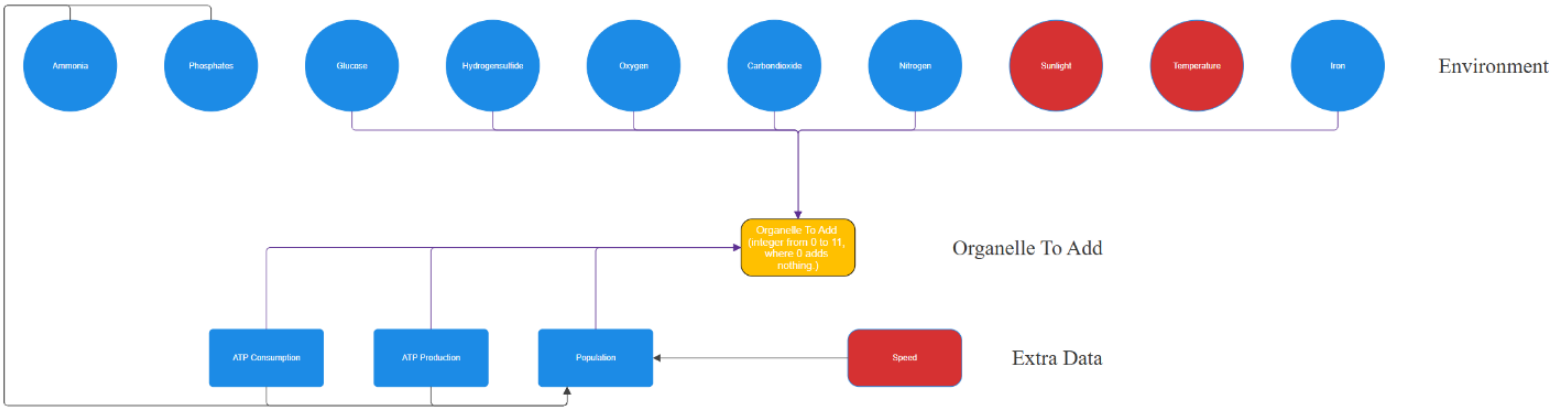

Figure 4: Bayes Net that compresses layer 2 into one node.

To reach the simplest model the connections between ATP Consumption, ATP Production, and Population with the Predictor node have been severed so that we have a minimum number of data points required of $2^6 = 64$. This is achievable within a couple of days and so will be done for the purposes of this project. It may be two separate graphs at this point, but it still functions well as a greedy-like approach. Each data point gathered will contain all information aside from the current organelles on Microbe, so this model can be upgraded to various versions of Figure 4's model, up to and including that exact model.

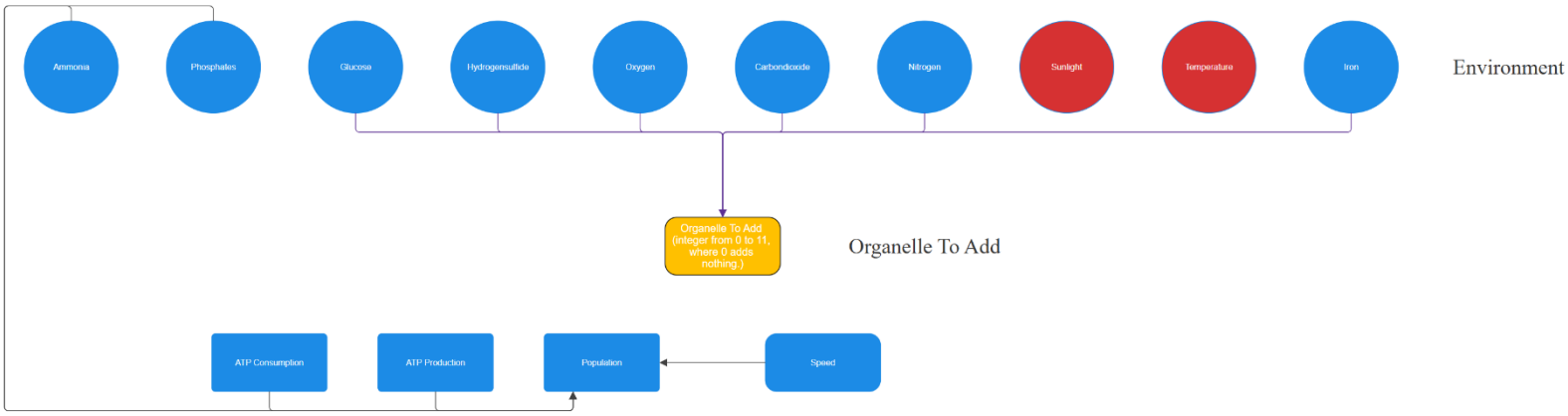

Figure 5: Final version of the Bayes Net model.

Deep Learning

The plan with the Deep Learning agent was to make a simple two inner/hidden layer, one input, one output layer neural network that would take in all of the features from its CSV file and output which action to take. The model is made of 26 features, two hidden layers of size 52, and an action layer of 13.

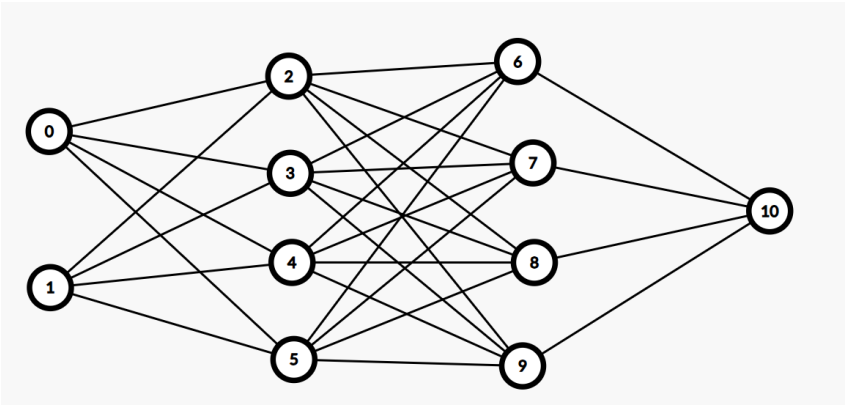

Figure 6: Simplified model of the Deep Learning neural network.

Figure 6 shows the simplified model of the neural network and the ratios therein. It does is not 1:1 as 146 nodes is a bit too much to show on a nice diagram.

Implementation

Controller

The controller (Thrive_AI.py) is imported by all agent files. It is the limbs and communicator of the agents, allowing them to interact with the game via reading and writing to files and taking control of the mouse and keyboard to do predetermined actions in any order the agent would like. It primarily uses pyautogui and pydirectinput to take control of the mouse and keyboard. Preferably it would be just pyautogui to limit the number of libraries being used but Thrive uses DirectX which ignores the virtual keys pyautogui uses, so pydirectinput must be used for the keyboard. From top down:

to_editor(): Clicks the editor button then waits for the file load.txt to say Loaded (provided by Game Modifications), before skipping through to the final page in the editor where the microbe can be modified, then returns control to the agent.

Convert_to_csv(): Converts log.txt into a csv file readable by the agent. Can overwrite/make a new a file or append the log.txt data to the end of an already existing csv file.

to_active_stage(): Clicks the “Confirm” button then clicks where an acknowledgement button would be if the cell does not produce enough ATP and will likely die off that round (usually happens when toxisome is picked near right away). It then waits for loading again before returning control to the agent.

select_part(): Selects an organelle in-game depending on the organelle name or number passed into the function.

add_part(): Adds the selected organelle to the microbe in a spiral. Surprisingly difficult to set up as it is on an unspecified hexagonal grid, but as Figure 10 shows, circles can be overlayed on each hexagonal ring, so with precise measurements and math I created a set of formulas with help from a very detailed and long website about hexagons to do so. The hexagons had a radius of about 162 pixels on a 4K monitor, but the optimal radius so that the multiples of that radius would have circles encompassing the 0th, 1st, 2nd, and 3rd rings was 132 pixels. There is no need to go past the 3rd ring as if all organelles locations are filled within the first three rings that would be 36 organelles plus the starting organelle in the center, which is well over the needed 20 generations.

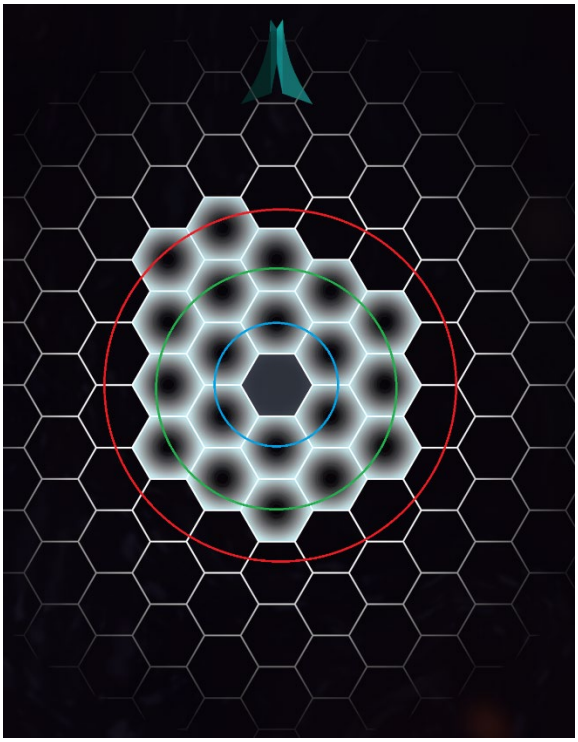

Figure 10: Proof that circles of approximate multiples of the same radius can encompass all needed rings.

test_position(): Outputs the current position of the mouse every second to the terminal so I can more easily find the pixel location of each button and decern the locations the organelles should be placed at on the microbe.

check_loading_completed(): Checks if load.txt contains “Loaded” as a line. Calls loading_complete() if so.

loading_complete(): Simply writes “NOT LOADED” into load.txt so check_loading_completed() does not think the game has loaded prematurely (basically resets the file, ready for the next loading screen.)

turn_on_cheats(): Turns on infinite compounds and unlimited growth cheats to allow the agent to near instantly be able to press the editor button. Needs to be run every time a save is loaded or a new game is created.

gather_data(): A whole section is dedicated to this.

Various tests are provided in the if __name__ == "__main__" section of the code, uncomment and use them as you like, though they are usually highly specific to what I needed them for and so may not be super useful.

Game Modifications

In order to facilitate data gathering so that the agents can read what the current statistics of the round/generation are, and so large chunks of data can be gathered for the Bayes Net and Deep Learning agents to train on, the game files needed to be modified as it does not provide log files to easily read from. It is a very long process to set up, but the process involved forking the game at version 1.0.0 from their GitHub, installing Godot, and searching through many of their files for anything related to environmental compounds or player species data. Unfortunately, the IDE I was using, VS Code, gave next to no info on the functions being used, where they were coming from, what files used them, etc. So I switched to Rider as it was recommended by the Thrive team, which allowed me to more quickly find what I needed.

I ended up modifying the MicrobeEditor file, in particular the OnEditorEnter() function so it sends all the data required when the editor opens. It gets the compound data from the current patch dataset and the player data (ATP Consumed, Produced, and Speed) from GetPlayerDataSource(). It formats and writes this data to "AI Player/log.txt", then queues an action to run a function I made called MarkLoaded(), which will run that function when a loading screen is encountered. The function overwrites a file called "AI Player/load.txt", writing "Loaded" in it to allow the Controller to know when loading is done and to resume. This function is also queued as an action to run when a loading screen is done when the active stage is entered to let the Controller know when it has loaded back into the editor.

Data Gathering

Data gathering went through many phases throughout making this project. It initially had the "selected" value, which would be used in training as the best option, be manually entered and be the "Organelle Suggestion" that appears in the editor (as seen in the bottom right of Figure 3.) However, this would end up making the Bayes Net and Deep Learning model be a proxy of this organelle suggester already in the game. So, I ended up making a program that would pick every possible organelle option, one at a time, running the game

and gathering the data from every possible future the program could take from the current generation, one generation into the future. Then, it would compare them to see which one is best, then set that as the selected organelle for the current generation, and continue to do that for each generation data is collected on. The algorithm needed to see future generation values as without that a regular mathematical algorithm could replace the AI agents. The algorithm for this also changed over time. What was initially select the organelle that leads to the highest population turned out to be a very greedy-like approach that inevitably lead the player species to die off in the long run, as it would prefer temporary gains over any long-term strategy. It also didn't help that after enough generations down a specific path, every choice seemed to lead to the same population outcome looking one generation ahead. This project turned into an algorithms project for a while as I tried to solve this problem. Eventually, I landed on the formula: if atp_production[i] - atp_consumption[i] + k*populations[i] > best, such that k = 0.001, set best to the output of that formula, and best_pos to i. This maximizes the difference between ATP production and consumption, while still considering the future population difference. k ended up being set to 0.001 as the difference between ATP production and consumption was typically in the single digits, and population was typically in the thousands. If the difference of ATP production and consumption were to rise into the second digit it would be expected that population would rise to the fifth digit at a similar rate, and so would remain balanced (this turned out to not always be the case if ATP production goes out of control and careens past 100 leaving consumption behind between 10-20, but was usually a safe bet.) The initial best was set such that i = 0 to avoid edge cases that could occur. best_pos, the numeric version of the selected organelle, is initially set to 12 which means to not pick any organelle, as the default should be to stick with what has worked thus far and not bloat the microbe any further if there is no preferable organelle to add. This algorithm does not go into the potential future of selecting thylakoid as that requires sunlight which will never reach the starting biome the player species is stuck in. This algorithm is then run as many times as needed to fill the data_log.csv file with enough data points. As I have learned through experience, it is best to not let the data gathering algorithm gather_data() in Thrive_AI.py for more than 15 or so generations, as the time it takes to load between the editor, active stage, and back to the editor takes more and more time, and so the cost of gathering the data greatly increases. That and because the amount of phosphate does not typically change between each generation, only each unique save/seed, it is more beneficial to run 10-15 generations per seed then make a new seed and run them again.

The data outputted to the CSV contains the current compounds and other local environmental factors: Ammonia, Glucose, Phosphates, Hydrogensulfide, Oxygen, Carbondioxide, Nitrogen, Sunlight, Temperature, Iron. It also includes current player

species data: Speed, ATP Production, ATP Consumption, and Population. All courtesy of the game modifications. It also contains the number of organelles currently on the microbe: cytoplasm, hydrogenase, metabolosomes, thylakoids, chemosynthesizing proteins, rusticyanin, nitrogenase, toxisome, flagellum, perforator pilus, chemoreceptor, slime jet. These are all manually kept track of by each agent and the gather_data() function. And finally, it can contain any additional info needed by the agent. For Bayes Net and Deep Learning this includes the "selected" column which tells which organelle is the best option to pick if being used for training, otherwise is used as the predictor/output of what organelle to select when being used.

As the Deep Learning model does not require nearly as many data points to start working, deciding what option is the best one for each run can be done manually, so I used the in-game's recommended option as the one to choose. This does not come with the advantage of future knowledge that the Bayes Net's data collection uses, but can still create very good, competitive microbes.

Numeric Planner (ENHSP)

Each action has a precondition that states it cannot have already selected that organelle and must have a total cost of less than or equal to 100 – the cost of that organelle. The parameters are any object that is mentioned in the preconditions and/or effects. The effects will always set the action's related organelle predicate to True, decrease the preferencer function by the number of organelles, and increase the number of organelles by one. It decreases the preferencer function by the number of organelles on the microbe as a larger microbe has much greater cost in terms of ATP usage, speed reduction, and the amount of compounds needed for mitosis. This means larger microbes are only beneficial if the environment has a supply of compounds and/or prey abundant enough to meet the needs of the larger species. So, to dynamically employ this I simply subtracted the current number of organelles each time a new organelle is added to the plan. Compound values are almost never above 100, and most of the organelle increases to preferencer have been balanced to be below or near a value of 20, and I have set a goal for the preferencer to be above or equal to 0 so if there are only negative options it will pick a plan that chooses no organelles to add, which is why this should work.

In order of actions in the file, this is the reasoning behind the preconditions and effects of each action:

Cytoplasm:

> Description: An organelle that simply has greater compound storage than the rest and produces ATP via glucose.

Cost: 22 MP so total is increased by 22.

Preferencer Increase: Initially the preferencer would only increase by the amount of glucose in the environment, but after considering the other organelles, I balanced it by adding 3.4, so preferencer is increased by glucose + 3.4. This is because when glucose was at 4.5 the expected total energy gathered was about equal with hydrogenase, and hydrogenase is about 76% faster at converting glucose to ATP than cytoplasm. So, values below this tick-over point are when cytoplasm's other properties outshine hydrogenase (like storage).

Hydrogenase:

Description: An organelle that specializes in converting glucose to ATP.

Cost: 45 MP so total is increased by 45.

Preferencer Increase: Preferencer is multiplied by 1.76 for the same balancing reason as the cytoplasm. Hydrogenase is 76% better when glucose levels are above 4.5, and as cytoplasm takes care of the offset, hydrogenase takes care of the multiplicative difference.

Metabolosomes:

Description: Metabolosomes are the best at converting glucose into ATP, but only when there is enough oxygen around as it is proportional to the concentration of oxygen.

Cost: 50 MP so total is increased by 50.

Preferencer Increase: At 32.2% oxygen I found metabolosomes to be 2.41 times better than hydrogenase, so with rounding I multiplied the result of glucose*oxygen/100 by 7.5 as 7.5 would be the likely difference when oxygen is at 100%. Preferencer is increased by the result.

Thylakoid:

Description: A simpler version of chloroplast, thylakoids produce glucose at a rate proportional to sunlight and carbon dioxide. Also produces some ATP with glucose.

Cost: 45 MP so total is increased by 45.

Preferencer Increase: Addition of glucose as it creates ATP via glucose a very similar rate to cytoplasm, but slightly slower (so no need to add offset like in cytoplasm). Also adds [the percentage of sunlight * (1 + the amount of CO_2)] * 15. * 15 as at 100% sunlight it is about 15* more efficient than cytoplasm, and that efficiency

comes from generating its own glucose rather than relying on external glucose which is why it is an addition to the external glucose rather than multiplication.

Chemosynthesizing proteins (chemproteins):

Description: Used to make glucose out of hydrogen sulfide then make ATP out of said glucose. Rate scales with CO_2 concentration.

Cost: 45 MP so total is increased by 45.

Preferencer Increase: The percentage of CO_2 * the amount of hydrogen sulfide. There is not much need to balance here as the amount of hydrogen sulfide is close to the amount of everything else.

Rusticyanin:

Description: Converts iron into ATP.

Cost: 45 MP so total is increased by 45.

Preferencer Increase: Just increased by the amount of iron. If there is more iron this is preferable, so no need to modify.

Nitrogenase:

Description: Converts ATP into ammonia which, if it has enough ATP for it, gives a great boost to population as ammonia is needed for mitosis.

Cost: 55 MP so total is increased by 55.

Preferencer Increase: (Glucose - Nitrogen * 2.676) * 2 as at 100% nitrogen it consumes 2.676 ATP per second. The goal of this preferencer increase is to make nitrogenase viable if enough ATP is being generated per second. As there is no internal ATP Production function currently, the next best thing was to use glucose as the placeholder as it is the most commonly used compound to make ATP out of. The * 2 is to make it competitive due to the subtraction involved.

Toxisome:

Description: Makes toxic chemicals to damage prey, making them easier to absorb or kill then consume.

Cost: 55 MP so total is increased by 55.

Preferencer Increase: Glucose + 3. It performs glycolysis (converting glucose into ATP) at the same rate as the cytoplasm but is also a very useful tool to gain energy

from other microbes, so a + 3 preference is added to make it more viable than cytoplasm when preconditions are met. It also costs a lot of ATP so should only be added after a good ATP-producing microbe is created.

Precondition Addition: In order to ensure it is not one of the first organelles to be added as that would kill the species, a requirement of having equal to or more than five organelles is added.

Flagellum:

Description: Allows the microbe to move much faster at the cost of a large sum of ATP while moving.

Cost: 55 MP so total is increased by 55.

Preferencer Increase: It is preferable when the microbe is giant and so cannot move very fast. As such, its preferencer increases by 1.5 * the number of organelles currently on the microbe.

Precondition Addition: In order to ensure this organelle is not added too early a precondition of there being equal to or greater than 10 organelles already on the microbe has been added.

Perforator Pilus (perforatorpilus):

Description: A big spike to pierce and damage prey. Useful to hunt other microbes and absorb their remains to gain ATP.

Cost: 30 PM so total is increased by 30.

Preferencer Increase: Arbitrary value of 10 was assigned to this as there is no compound that would affect it and it does not require more ATP to operate than any other organelle. 10 was chosen as it is near to the expected values of the other organelles, but low enough so that if there is a “must pick this” organelle that will be picked first. It was also picked so that the total preference would be equal to five if the microbe were to have five organelles upon selection. Five was the value to aim for, but do not want it to select this option over not picking anything if it is too large of a microbe so it could not be related to the number of organelles on the microbe.

Precondition Addition: In order to ensure this organelle is not picked too early, a minimum of five organelles is required to be on the microbe.

Chemoreceptor:

Description: The ability for a microbe to “smell” points it in the direction of any assigned compound, but for our purposes that compound is glucose.

Cost: 45 MP so total is increased by 45.

Preferencer Increase: The amount of glucose + 5. This is because while it is useful to find glucose, if there is almost no glucose left in the biome this should not be added, but if there is some and the microbe already has some glucose processing organelles, it is preferable to have one.

Precondition Addition: Must have greater than or equal to five organelles already on the microbe as this does not directly generate ATP so need to prevent it being taken too early. Also, a condition that there needs to be less than or equal to 1 quantity of glucose in the biome. This is because it is only really useful if a microbe does not already stumble into glucose clouds frequently and needs to travel greater distances.

Slimejet:

Description: A booster that shoots slime to quickly propel a microbe forward. Useful to escape predators or catch prey.

Cost: 60 MP so total is increased by 60.

Preferencer Increase: Similarly to flagellum, it is only really useful when the microbe has a larger body. However, unlike flagellum slimejets do not come at a great ATP cost. This is why preferencer is increased by the number of organelles * 1.6, as it is typically preferable over flagellum.

Precondition Addition: Also a restriction that there must be equal to or greater than 5 organelles on the microbe already before this is picked. It is less than the flagellum as it does not come with the same extreme ATP cost so it is more likely the microbe can afford this if played optimally up to this point.

Bayes Net

The Bayes Net was created with the same final model as shown in the Methods section. It uses a LinearGaussianBayesianNetwork as with it the weights do not need to be manually set, rather trained on a curated dataset made earlier. After it is trained on the dataset, it runs like the other agents: Go to the editor, convert the log.txt to a csv file, read the csv file,

predict what the next organelle to be added should be by dropping the selected column and predicting what it should be given the other variables, select the corresponding organelle if it is a value between 0 and 11 inclusive then add said organelle, otherwise select and add nothing, then finally go back to the active stage to do it again for a total of 20 generations. As the Bayes Net predicts what the value should be in a linear way, it will almost never pick a discrete value, so it is rounded using Python's built-in round() function to convert it into a usable discrete decision that matches the original output as close as possible.

Deep Learning

Intuition due to my history with neural networks and especially deep learning and reinforcement learning carried a lot of the reasoning behind the initial values in the model.

To start, there are 26 features as there are 10 compounds, 3 player species variables, 12 types of organelles that the microbe can be made of, and the current population of the player species. From my experience, a good starting point is to have the hidden layers (if there is just one or two) be double the size of the features, so both hidden layers are 52 nodes tall/wide. The agent can select any of the 12 types of organelles to add or choose to not add any (can only pick one option, not multiple organelles as though the output layer would still be finite, it would be too large and so require too many data points in training), so the output layer is 13 nodes wide.

To train the model, it runs with a test size of 15% and a learning rate of 1% over the course of 10 epochs. This test size was chosen as it is within a widely accepted range of test sizes and the number of data points to train from is large enough. Though I typically increase the learning rate up to 10%, I found through initial testing that the 1% learning rate works surprisingly well. In reality 10 epochs is likely slightly overfitting the model as the size of the dataset is not that large yet (not even above 100), but testing has shown that despite this it still works for at least the starting 8 generations very well.

After training, when started it will turn on the cheats like normal, go to the editor, then convert the current data from log.txt to deep_learning_log.csv which it will then drop the selected column from and make a list out of only the current generation's data, converting said list into tensor floats. Then, it will send that data through the neural network, outputting its selection. If it is != 12 it will select the part and add it, else it will skip that step, then either way go to the active stage and then back to the editor. It will loop this process from converting the data to a csv file onward for 20 generations.

To see if it succeeded the csv file must be checked for its population. If it ever reached 50 it died, and if the final population is equal to or above 300 it succeeded!

Results

Numeric Planner (ENHSP)

I was unfortunately not able to get ENHSP to work with my numeric planner. Though the domain compiles my problem file does not. I have tried a few different ways to get it to work but to no avail. I got far with setting it up, though. I made a local server to parse it in VS Code, but it got stuck on detecting packages, so I tried manually running it in planutils in WSL after installing JDK-17, however neither ENHSP nor ENHSP-2020 ended up working. I do see some flaws in the planner which I discuss solutions for some of them in the What Would I Change section of this report, but I think overall it would be an effective agent.

Bayes Net

The Bayes Net still does not have enough data points to run well, so tended to pick options that did not make sense in the moment like metabolosomes even though there was no or next to no oxygen at the beginning for it to work effectively. However, it did survive at least 10 generations in my last test run, so I would say it is still an overall success!

Back when the Bayes Net was using the initial 2^{12} connections with one node model that was in the demo, it would keep selecting cytoplasm even though it was making a giant microbe that was slowly killing it the larger it got (Figure 11), so it is now much better than the demo at least! And definitely more varied in its choices which shows it did learn from the data.

Deep Learning

Deep learning ended up being the most successful out of all of them, getting populations above 10000 and even 30000 sometimes. It suffered from the same fate as the other two, though, being that it could not remove any organelles and so could not adapt if a resource that was very beneficial to adapt to consuming became nearly completely diminished, making those organelles more of a burden than a help.

How To Run It

Currently the agents can only be run on 4K 16:9 screens as I have not yet found a way to easily check the resolution and aspect ratio of the user's monitor, and the game can change proportions on differently sized screens, which I cannot easily test for and put alternative locations for with each monitor size. If time permits after submitting this project I will update the GitHub project with a converter so it can work on anyone's device.

All agents have a time.sleep() function of 2 seconds to allow the user to quickly open the game before the agent begins running and takes control of the mouse and keyboard. If during the run you would like to stop the agent for any reason quickly move your mouse to any corner of your screen

What Would I Change

In the future given much greater time I would find a way to output the total energy collected in a run and use that to determine which option is the best one when running the data gathering algorithm. I would also like to run the algorithm for long enough to be able to use the entire Bayes Net effectively, and use it as the training dataset for the deep learning algorithm. I would also like to make a set of functions for the planner that handle the internal usage and production of compounds as organelles like hydrogenase are only really useful in the long run if you have organelles that produce an excess of glucose, so the current planner and the other agents do not currently cover this.

Chemosynthesizing Proteins are almost always the optimal option to add when the amount of hydrogen sulfide is >= 11. So balancing the actions in the planner such that this is the case would be preferred.

If there were time and ENHSP loaded the domain and problem file, I would have done a lot more balancing and optimizations of the actions. In particular making an additional function for each organelle currently on the microbe so that the specialized cells could only be picked up to one time each, as there is almost never a need for more than one.